Images



© Copyright 2004-2026, Bootlin
Creative Commons BY-SA 3.0
Corrections, suggestions, contributions and translations are welcome!



Introduction to images



Overview 1/3

- An `image` is the top level recipe and is used alongside the `machine` definition.
- Whereas the `machine` describes the hardware used and its capabilities, the `image` is architecture agnostic and defines how the root filesystem is built, with what packages.
- By default, several images are provided in openembedded-core:
  - `meta*/recipes*/images/*.bb`



Overview 2/3

- Here are a few common images:

core-image-base Console-only image, with full support of the hardware.
core-image-minimal Small image, capable of booting a device.
core-image-minimal-dev Small image with extra tools, suitable for development.
core-image-x11 Image with basic X11 support.
core-image-weston Image with basic Wayland support.
core-image-rt Like `core-image-minimal` with real time tools and test suite.



Overview 3/3

- An `image` is no more than a recipe.
- It has a description, a license (optional) and inherits the `core-image` class.



Organization of an image recipe

Some special configuration variables are used to describe an image:

- IMAGE_BASENAME: The name of the output image files. Defaults to `${PN}`.
- IMAGE_INSTALL: List of packages and package groups to install in the generated image (only toplevel packages, dependencies unnecessary)
- IMAGE_ROOTFS_SIZE: The final root filesystem size.
- IMAGE_FEATURES: List of features to enable in the image (e.g. `allow-root-login`).
- IMAGE_FSTYPES: List of formats the OpenEmbedded build system will use to create images. Could be set in machine definitions too (machine dependent).
- IMAGE_LINGUAS: List of the locales to be supported in the image.
- IMAGE_PKGTYPE: Package type used by the build system. One of `deb`, `rpm` and `ipk`.
- IMAGE_POSTPROCESS_COMMAND: Shell commands to run at post process.
- EXTRA_IMAGEDEPENDS: Recipes to be built with the image, but which do not install anything in the root filesystem (e.g. the bootloader).



Example of an image

```
SUMMARY = "Example image"
IMAGE_INSTALL = "packagegroup-core-boot dropbear ninvaders"
IMAGE_LINGUAS = " "

inherit core-image
```

Note: unlike other recipes, image recipes don't need to set LICENSE.



Root filesystem generation

- Image generation overview:
  1. An empty directory is created for the root filesystem.
  2. Packages from IMAGE_INSTALL are installed into it using the package manager.
  3. One or more images files are created, depending on the IMAGE_FSTYPES value.
- Root filesystem creation is specific to the IMAGE_PKGTYPE value. It should be defined in the image recipe, otherwise the first valid package type defined in PACKAGE_CLASSES is used.
- All the magic is done in `meta/classes-recipe/rootfs_${IMAGE_PKGTYPE}.bbclass`



Image types



IMAGE_FSTYPES

- Configures the resulting root filesystem image format.
- If more than one format is specified, one image per format will be generated.
- Image formats instructions are provided by `openembedded-core`, in `meta/classes-recipe/image_types.bbclass`
- Common image formats are: `ext2`, `ext3`, `ext4`, `squashfs`, `squashfs-xz`, `cpio`, `jffs2`, `ubifs`, `tar.bz2`, `tar.gz`…



Creating an image type

- If you have a particular layout on your storage (for example bootloader location on an SD card), you may want to create your own image type.
- This is done through a class that inherits from `image_types`.
- It has to define a function named `IMAGE_CMD:<type>`.
- Append it to IMAGE_TYPES



Creating an image conversion type

- Common conversion types are: `gz`, `bz2`, `sha256sum`, `bmap`…
- This is done through a class that inherits from `image_types`.
- It has to define a function named CONVERSION_CMD:<type>.
- Append it to `CONVERSIONTYPES`
- Append valid combinations to IMAGE_TYPES



wic

- `wic` is a tool that can create a flashable image from the compiled packages and artifacts.
- It can create partitions (but doesn't support raw flash partitions and filesystems)
- It can select which files are located in which partition through the use of plugins.
- The final image layout is described in a `.wks` or `.wks.in` file.
- It can be extended in any layer.
- Usage example:

```
WKS_FILE = "imx-uboot-custom.wks.in"
IMAGE_FSTYPES = "wic.bmap wic"
```

- Note: bmaptool is an alternative to `dd`, skipping uninitialized contents in partitions.



imx-uboot-custom.wks.in

```
part u-boot --source rawcopy --sourceparams="file=imx-boot" --no-table --align ${IMX_BOOT_SEEK}
part /boot --source bootimg-partition --use-uuid --fstype=vfat --label boot --active --align 8192 --size 64
part / --source rootfs --use-uuid --fstype=ext4 --label root --exclude-path=home/ --exclude-path=opt/ --align 8192
part /home --source rootfs --rootfs-dir=${IMAGE_ROOTFS}/home --use-uuid --fstype=ext4 --label home --align 8192
part /opt --source rootfs --rootfs-dir=${IMAGE_ROOTFS}/opt --use-uuid --fstype=ext4 --label opt --align 8192

bootloader --ptable msdos
```

- Copies `imx-boot` from `$DEPLOY_DIR_IMAGE` in the image, aligned on (and so at that offset) `${IMX_BOOT_SEEK}`.
- Creates a first partition, formatted in FAT32, with the files listed in the `IMAGE_BOOT_FILES` variable.
- Creates an `ext4` partition with the contents on the root filesystem, excluding the content of `/home` and `/opt`
- Creates two `ext4` partitions, one with the content of `/home`, the other one with the content of `/opt`, from the image root filesystem.



Package groups



Overview

- Package groups are a way to group packages by functionality or common purpose.
- Package groups are used in image recipes to help building the list of packages to install.
- A package group is yet another recipe.
  - Using the `packagegroup` class.
  - The generated binary packages do not install any file, but they require other packages.
- Be careful about the PACKAGE_ARCH value:
  - Set to the value `all` by default,
  - Must be explicitly set to `${MACHINE_ARCH}` when there is a machine dependency.



Common package groups

- `packagegroup-base`
  - Adds many core packages to the image based on MACHINE_FEATURES and DISTRO_FEATURES
- `packagegroup-core-boot`
- `packagegroup-core-buildessential`
- `packagegroup-core-nfs-client`
- `packagegroup-core-nfs-server`
- `packagegroup-core-tools-debug`
- `packagegroup-core-tools-profile`



Example

./meta/recipes-core/packagegroups/packagegroup-core-tools-debug.bb:

```
SUMMARY = "Debugging tools"

inherit packagegroup

RDEPENDS:${PN} = "\
    gdb \
    gdbserver \
    strace"
```

Practical lab - Create a custom image



- Write an image recipe
- Choose the packages to install